In [57]:
```python
import pandas as pd
from pathlib import Path
import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gaussian_kde, skew, kurtosis, norm
from scipy.stats import pearsonr
import statsmodels.api as sm
import seaborn as sns
from statsmodels.nonparametric.smoothers_lowess import lowess
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
from linearmodels.panel import RandomEffects
from linearmodels.panel import PanelOLS
import statsmodels.formula.api as smf
from statsmodels.tsa.stattools import adfuller
from statsmodels.stats.diagnostic import acorr_breusch_godfrey
from statsmodels.tsa.stattools import coint
from statsmodels.tsa.vector_ar.vecm import coint_johansen
```

In [21]:
```python
final_data_path = Path("/Users/lorenzouberti/Desktop/Panel-Data-Project-Financial-Econometrics/Data")
df_Final = pd.read_excel(final_data_path / "final_panel_dataset.xlsx")

df_Final = df_Final.sort_values(["country", "year"])
df_Final = df_Final[['country', 'year', 'gdppc', 'labour_productivity', 'cpi', 'trade_openness', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']]
df_Final = df_Final.sort_values(["country", "year"])

# Logging GDP per capita first
df_Final['ln_gdppc'] = np.log(df_Final['gdppc'])
df_Final["ln_gdppc_lag_2"] = df_Final.groupby("country")["ln_gdppc"].shift(2)
# Creating the 5 year lagged GDP per capita growth variable
df_Final["ln_gdp_growth_2y"] = df_Final["ln_gdppc"] - df_Final["ln_gdppc_lag_2"]

# Doing of rthe alternative dependent variable: labour productivity growth
df_Final['ln_labour_productivity'] = np.log(df_Final['labour_productivity'])
df_Final["ln_labour_productivity_lag2"] = df_Final.groupby("country")["ln_labour_productivity"].shift(2)
df_Final["ln_labour_productivity_growth_2y"] = df_Final["ln_labour_productivity"] - df_Final["ln_labour_productivity_lag2"]

# taking teh logarithm of the other variables as well
df_Final['ln_cpi'] = np.log(df_Final['cpi'])
df_Final['ln_trade_openness'] = np.log(df_Final['trade_openness'])

variables = [ 'ln_gdppc','ln_labour_productivity', 'ln_cpi', 'ln_trade_openness', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp', "ln_gdp_growth_2y", "ln_labour_productivity_growth_2y" ]

y = 'ln_gdp_growth_2y'
X = ['liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']
Con = ['ln_cpi', 'ln_trade_openness']

# Splitting my data into the seperate regimes -> pre 1928, 1929-1971, post 1971

df_Regime = df_Final.copy()

conditions = [
        (df_Regime["year"] < 1928),
        (df_Regime["year"] >= 1928) & (df_Regime["year"] <= 1971),
        (df_Regime["year"] >= 1974)
]
regimes = ['Gold Standard', 'Bretton Woods','Finanzialisation']

regime_arr = np.select(conditions, regimes, default='')
df_Regime['regime'] = pd.Series(regime_arr, index=df_Regime.index).replace('', np.nan)

df_GS = df_Regime[df_Regime['regime'] == 'Gold Standard']
df_BW = df_Regime[df_Regime['regime'] == 'Bretton Woods']
df_Fin = df_Regime[df_Regime['regime'] == 'Finanzialisation']
```

In [25]:
```python
import numpy as np
from scipy import stats
import statsmodels.formula.api as smf

def poolability_test(df, y, x, controls, entity):

    rhs_vars = x + controls
    rhs_common = " + ".join(rhs_vars)

    # Restricted
    restricted_formula = f"{y} ~ {rhs_common} + C({entity})"

    # Unrestricted
    interactions = " + ".join([f"{var}:C({entity})" for var in rhs_vars])
    unrestricted_formula = (
        f"{y} ~ {rhs_common} + C({entity}) + {interactions}"
    )

    #  estimate models
    m_r = smf.ols(restricted_formula, data=df).fit()
    m_u = smf.ols(unrestricted_formula, data=df).fit()

    # SSR
    SSR_r = np.sum(m_r.resid ** 2)
    SSR_u = np.sum(m_u.resid ** 2)

    # dof
    q = int(m_u.df_model - m_r.df_model)   # number of restrictions
    df_u = int(m_u.df_resid)

    # F star
    F = ((SSR_r - SSR_u) / q) / (SSR_u / df_u)
    p_value = 1 - stats.f.cdf(F, q, df_u)

    return {
        "Restricted formula": restricted_formula,
        "Unrestricted formula": unrestricted_formula,
        "F statistic": F,
        "p-value": p_value,
        "q restrictions": q,
        "df residual (unrestricted)": df_u
    }

y = "ln_gdp_growth_2y"
X = ['liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']
Con = ['ln_cpi', 'ln_trade_openness']

results = {}

for name, df_sub in {
    "Gold Standard (GS)": df_GS,
    "Bretton Woods (BW)": df_BW,
    "Financialization (FIN)": df_Fin
}.items():

    res = poolability_test(
        df=df_sub,
        y=y,
        x=X,
        controls=Con,
        entity="country"
    )

    results[name] = res

    print(f"\n{name}")
    print("-" * 50)
    print(f"F-statistic : {res['F statistic']:.3f}")
    print(f"p-value     : {res['p-value']:.4f}")
    print(f"Restrictions: {res['q restrictions']}")
```

```

Gold Standard (GS)
--------------------------------------------------
F-statistic : 2.753
p-value     : 0.0000
Restrictions: 40

Bretton Woods (BW)
--------------------------------------------------
F-statistic : 5.474
p-value     : 0.0000
Restrictions: 40

Financialization (FIN)
--------------------------------------------------
F-statistic : 3.365
p-value     : 0.0000
Restrictions: 40
```

In [33]:
```python
# Adf Test for stationarity

def adf_test(series, maxlag=None, regression = 'c'):

    series = series.dropna()
    if len(series) < 10:
        return None
    result = adfuller(series, maxlag=maxlag, regression=regression, autolag="AIC")
    return {
        "ADF statistic": result[0],
        "p-value": result[1],
        "used lags": result[2],
        "nobs": result[3],
        "5% crit": result[4]["5%"],
        "stationary_5pct": result[1] < 0.05
    }

vars_to_test = [
    "ln_gdp_growth_2y",
    "liquid_liabilities",
    "bank_loans_gdp",
    "mcap_gdp",
    "ln_cpi",
    "ln_trade_openness"
]

def run_adf_by_country(df, variables, regime_name):
    results = []

    for country in df["country"].unique():
        df_c = df[df["country"] == country]

        for var in variables:
            res = adf_test(df_c[var], regression="c")
            if res is None:
                continue

            results.append({
                "regime": regime_name,
                "country": country,
                "variable": var,
                "ADF stat": res["ADF statistic"],
                "p-value": res["p-value"],
                "stationary (5%)": res["stationary_5pct"],
                "nobs": res["nobs"]
            })

    return pd.DataFrame(results)

adf_GS  = run_adf_by_country(df_GS,  vars_to_test, "GS")
adf_BW  = run_adf_by_country(df_BW,  vars_to_test, "BW")
adf_FIN = run_adf_by_country(df_Fin, vars_to_test, "FIN")

adf_results = pd.concat([adf_GS, adf_BW, adf_FIN], ignore_index=True)

adf_results.head()
```

Out[33]:

	regime	country	variable	ADF stat	p-value	stationary (5%)	nobs
0	GS	Australia	ln_gdp_growth_2y	-3.126785	0.024633	True	43
1	GS	Australia	liquid_liabilities	-2.948312	0.040011	True	41
2	GS	Australia	bank_loans_gdp	-0.527711	0.886494	False	45
3	GS	Australia	mcap_gdp	-2.288818	0.175600	False	28
4	GS	Australia	ln_cpi	-0.583640	0.874636	False	42

In [34]:
```python
stationarity_by_var = (
    adf_results
    .groupby("variable")["stationary (5%)"]
    .mean()
    .reset_index()
    .rename(columns={"stationary (5%)": "share_stationary"})
)

stationarity_by_var["share_nonstationary"] = 1 - stationarity_by_var["share_stationary"]

print(stationarity_by_var)
```

```
             variable  share_stationary  share_nonstationary
0      bank_loans_gdp          0.000000             1.000000
1  liquid_liabilities          0.062500             0.937500
2              ln_cpi          0.270833             0.729167
3    ln_gdp_growth_2y          0.500000             0.500000
4   ln_trade_openness          0.125000             0.875000
5            mcap_gdp          0.166667             0.833333
```

In [ ]:
```python
#serial Autocorrelation test

def bg_test_one_country(df_country, y, x, controls, nlags=2):
    rhs = " + ".join(x + controls)
    formula = f"{y} ~ {rhs}"


    model = smf.ols(formula, data=df_country).fit()

    # Breusch-Godfrey test
    lm_stat, lm_pvalue, f_stat, f_pvalue = acorr_breusch_godfrey(model, nlags=nlags)

    return {
        "LM stat": lm_stat,
        "LM p-value": lm_pvalue,
        "F stat": f_stat,
        "F p-value": f_pvalue,
        "nobs": int(model.nobs)
    }

def bg_test_by_regime(df_regime, regime_name, y, x, controls, nlags=2, min_obs=15):
    results = []

    for country in sorted(df_regime["country"].unique()):
        df_c = df_regime[df_regime["country"] == country].dropna(subset=[y] + x + controls)

        # skip if too few observations
        if len(df_c) < min_obs:
            continue

        out = bg_test_one_country(df_c, y, x, controls, nlags=nlags)
        results.append({
            "regime": regime_name,
            "country": country,
            "nlags": nlags,
            **out
        })

    return pd.DataFrame(results)

y = "ln_gdp_growth_2y"
X = ['liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']
Con = ['ln_cpi', 'ln_trade_openness']

# oicking the lag order
nlags = 2

bg_GS  = bg_test_by_regime(df_GS,  "GS",  y, X, Con, nlags=nlags)
bg_BW  = bg_test_by_regime(df_BW,  "BW",  y, X, Con, nlags=nlags)
bg_FIN = bg_test_by_regime(df_Fin, "FIN", y, X, Con, nlags=nlags)

bg_results = pd.concat([bg_GS, bg_BW, bg_FIN], ignore_index=True)

# Flag rejection of no autocorrelation
bg_results["reject_no_autocorr_5pct"] = bg_results["LM p-value"] < 0.05

# summary
summary_bg = (
    bg_results.groupby("regime")["reject_no_autocorr_5pct"]
    .mean()
    .reset_index(name="share_reject_BG_5pct")
)

print(summary_bg)

# see dem p value
print(bg_results.sort_values(["regime", "LM p-value"]).head())
print(bg_results)
```

```
  regime  share_reject_BG_5pct
0     BW              1.000000
1    FIN              1.000000
2     GS              0.888889
   regime         country  nlags    LM stat    LM p-value     F stat  \
9      BW       Australia      2  29.818532  3.349563e-07  37.847532
10     BW          Canada      2  26.829070  1.493281e-06  28.124467
16     BW  United Kingdom      2  25.064821  3.607806e-06  23.826909
17     BW   United States      2  20.264650  3.977289e-05  15.367951
12     BW         Germany      2  18.438536  9.911121e-05  12.984141

      F p-value  nobs  reject_no_autocorr_5pct
9   1.408748e-09    44                     True
10  4.406976e-08    44                     True
16  2.562720e-07    44                     True
17  1.496144e-05    44                     True
12  5.680608e-05    44                     True
   regime         country  nlags    LM stat    LM p-value     F stat  \
0      GS       Australia      2   9.482135  8.729325e-03   5.101091
1      GS          Canada      2  12.817247  1.647291e-03   8.316328
2      GS         Denmark      2   9.907001  7.058658e-03   5.289752
3      GS         Germany      2  10.070108  6.505846e-03   5.408242
4      GS          Norway      2   4.683625  9.615320e-02   2.108704
5      GS          Sweden      2   7.824262  1.999784e-02   3.889348
6      GS     Switzerland      2   6.355874  4.167153e-02   3.012822
7      GS  United Kingdom      2  16.338233  2.832681e-04  11.313643
8      GS   United States      2   8.730515  1.271138e-02   4.522582
9      BW       Australia      2  29.818532  3.349563e-07  37.847532
10     BW          Canada      2  26.829070  1.493281e-06  28.124467
11     BW         Denmark      2  15.899787  3.526996e-04  10.184840
12     BW         Germany      2  18.438536  9.911121e-05  12.984141
13     BW          Norway      2   9.512039  8.599772e-03   4.964536
14     BW          Sweden      2  17.734827  1.409066e-04  12.153999
15     BW     Switzerland      2  15.381977  4.569263e-04   9.674867
16     BW  United Kingdom      2  25.064821  3.607806e-06  23.826909
17     BW   United States      2  20.264650  3.977289e-05  15.367951
18    FIN       Australia      2  13.741450  1.037725e-03   8.218978
19    FIN          Canada      2  17.865961  1.319641e-04  12.439478
20    FIN         Denmark      2  14.642900  6.612026e-04   9.036564
21    FIN         Germany      2   9.005892  1.107632e-02   4.636189
22    FIN          Norway      2  24.570773  4.618750e-06  23.331881
23    FIN          Sweden      2  11.261491  3.585901e-03   6.209368
24    FIN     Switzerland      2  18.423299  9.986919e-05  13.118430
25    FIN  United Kingdom      2  17.089328  1.945806e-04  11.542087
26    FIN   United States      2  15.116786  5.217129e-04   9.487564

      F p-value  nobs  reject_no_autocorr_5pct
0   1.564445e-02    29                     True
1   2.187114e-03    29                     True
2   1.077005e-02    38                     True
3   9.869266e-03    38                     True
4   1.390312e-01    38                    False
5   3.148579e-02    38                     True
6   6.421591e-02    38                     True
7   2.180992e-04    38                     True
8   2.326320e-02    29                     True
9   1.408748e-09    44                     True
10  4.406976e-08    44                     True
11  3.123379e-04    44                     True
12  5.680608e-05    44                     True
13  1.247002e-02    44                     True
14  9.261826e-05    44                     True
15  4.338756e-04    44                     True
16  2.562720e-07    44                     True
17  1.496144e-05    44                     True
18  1.185021e-03    43                     True
19  8.296445e-05    43                     True
20  6.853047e-04    43                     True
21  1.636311e-02    43                     True
22  3.636442e-07    43                     True
23  4.921145e-03    43                     True
24  5.603571e-05    43                     True
25  1.413119e-04    43                     True
26  5.102717e-04    43                     True
```

In [43]:
```python
# testing to see if there are some structural breask int tyeh data set

formula = f"{y} ~ " + " + ".join(X + Con)


def chow_test_known_break(df_country, formula, break_year):

    df_country = df_country.dropna().copy()
    df_country = df_country.sort_values("year")

    df_pre  = df_country[df_country["year"] <= break_year]
    df_post = df_country[df_country["year"] > break_year]

    # Need enough obs on both sides
    if len(df_pre) < 10 or len(df_post) < 10:
        return None

    m_full = smf.ols(formula, data=df_country).fit()
    m_pre  = smf.ols(formula, data=df_pre).fit()
    m_post = smf.ols(formula, data=df_post).fit()

    SSR_full = float(np.sum(m_full.resid**2))
    SSR_pre  = float(np.sum(m_pre.resid**2))
    SSR_post = float(np.sum(m_post.resid**2))

    k = int(m_full.df_model + 1)  # number of parameters incl intercept
    n = int(m_full.nobs)

    # Chow F-stat
    num = (SSR_full - (SSR_pre + SSR_post)) / k
    den = (SSR_pre + SSR_post) / (n - 2*k)
    if den <= 0:
        return None

    F = num / den
    p = 1 - stats.f.cdf(F, k, n - 2*k)

    return {"break_year": break_year, "F": F, "p_value": p, "n": n, "k": k}

breaks_by_regime = {
    "GS":  [1893, 1907, 1914],
    "BW":  [1933, 1945, 1958],
    "FIN": [1987, 2001, 2008],
}

def chow_tests_regime(df_regime, regime_name, formula, break_years):
    out = []
    for country in sorted(df_regime["country"].unique()):
        df_c = df_regime[df_regime["country"] == country].copy()
        for by in break_years:
            res = chow_test_known_break(df_c, formula, by)
            if res is None:
                continue
            out.append({"regime": regime_name, "country": country, **res})
    return pd.DataFrame(out)

chow_GS  = chow_tests_regime(df_GS,  "GS",  formula, breaks_by_regime["GS"])
chow_BW  = chow_tests_regime(df_BW,  "BW",  formula, breaks_by_regime["BW"])
chow_FIN = chow_tests_regime(df_Fin, "FIN", formula, breaks_by_regime["FIN"])

chow_results = pd.concat([chow_GS, chow_BW, chow_FIN], ignore_index=True)

# Flag significant breaks
chow_results["reject_stability_5pct"] = chow_results["p_value"] < 0.05

# Summary table: share of (country × breakdate) tests rejecting stability
chow_summary = (
    chow_results.groupby("regime")["reject_stability_5pct"]
    .mean().reset_index(name="share_reject_chow_5pct")
)
print(chow_summary)

# (Optional) see the strongest breaks
print(chow_results.sort_values("p_value").head(15))
```

```
  regime  share_reject_chow_5pct
0     BW                0.611111
1    FIN                0.722222
2     GS                0.400000
   regime         country  break_year          F       p_value   n  k  \
17     BW          Canada        1945  14.082841  8.839473e-08  44  6
45    FIN     Switzerland        1987  10.841699  1.711632e-06  43  6
31     BW   United States        1945   8.018613  2.505816e-05  44  6
19     BW         Denmark        1945   7.861060  2.986636e-05  44  6
46    FIN     Switzerland        2001   7.083086  8.142059e-05  43  6
13     GS  United Kingdom        1914   7.705412  1.277131e-04  35  6
29     BW  United Kingdom        1945   6.509265  1.456766e-04  44  6
25     BW          Sweden        1945   6.063597  2.539212e-04  44  6
49    FIN   United States        1987   4.982212  1.124784e-03  43  6
33    FIN       Australia        1987   4.962267  1.155454e-03  43  6
32     BW   United States        1958   4.807013  1.342247e-03  44  6
50    FIN   United States        2001   4.693941  1.665748e-03  43  6
15     BW       Australia        1945   4.287258  2.795966e-03  44  6
42    FIN          Norway        2001   4.117362  3.747029e-03  43  6
11     GS     Switzerland        1914   4.444258  3.987821e-03  35  6

    reject_stability_5pct
17                   True
45                   True
31                   True
19                   True
46                   True
13                   True
29                   True
25                   True
49                   True
33                   True
32                   True
50                   True
15                   True
42                   True
11                   True
```

In [53]:
```python
y = "ln_gdp_growth_2y"

finance_vars = ["liquid_liabilities", "bank_loans_gdp", "mcap_gdp"]
controls = ["ln_cpi", "ln_trade_openness"]
base_rhs = finance_vars + controls

def add_break_dummies(df):
    df = df.copy()
    df["D_1914"] = (df["year"] >= 1914).astype(int)
    df["D_1945"] = (df["year"] >= 1945).astype(int)
    df["D_1987"] = (df["year"] >= 1987).astype(int)
    return df

df_GS  = add_break_dummies(df_GS)
df_BW  = add_break_dummies(df_BW)
df_FIN = add_break_dummies(df_Fin)

def country_hac_regressions(df, regime, breaks, maxlags=2, min_obs=15):
    rows = []
    rhs = base_rhs + breaks
    formula = f"{y} ~ " + " + ".join(rhs)

    # columns required for estimation (so we can dropna explicitly)
    required = [y] + rhs + ["country", "year"]

    for c in sorted(df["country"].unique()):
        df_c = df[df["country"] == c].copy()

        # Drop missing *before* calling statsmodels
        df_c = df_c.dropna(subset=required)

        # Need enough obs after dropping missing
        if len(df_c) < min_obs:
            continue

        # HAC maxlags must be < number of observations (rule of thumb)
        ml = min(maxlags, max(1, len(df_c)//4))

        try:
            model = smf.ols(formula, data=df_c).fit(
                cov_type="HAC",
                cov_kwds={"maxlags": ml}
            )
        except Exception as e:
            # Keep track of failures rather than crashing
            rows.append({
                "regime": regime,
                "country": c,
                "variable": "__MODEL_FAILED__",
                "coef": np.nan,
                "se": np.nan,
                "t": np.nan,
                "p": np.nan,
                "error": str(e),
                "nobs": len(df_c)
            })
            continue

        for v in finance_vars:
            rows.append({
                "regime": regime,
                "country": c,
                "variable": v,
                "coef": model.params.get(v, np.nan),
                "se": model.bse.get(v, np.nan),
                "t": model.tvalues.get(v, np.nan),
                "p": model.pvalues.get(v, np.nan),
                "nobs": int(model.nobs),
                "maxlags_used": ml
            })

    return pd.DataFrame(rows)

res_GS  = country_hac_regressions(df_GS,  "GS",  breaks=["D_1914"])
res_BW  = country_hac_regressions(df_BW,  "BW",  breaks=["D_1945"])
res_FIN = country_hac_regressions(df_Fin, "FIN", breaks=["D_1987"])

results = pd.concat([res_GS, res_BW, res_FIN], ignore_index=True)

# Optional: see if any country models failed
failed = results[results["variable"] == "__MODEL_FAILED__"]
print("Model failures:", len(failed))
if len(failed) > 0:
    print(failed[["regime","country","nobs","error"]].head(10))

# Keep only the finance coefficient rows for hypothesis testing
results_finance = results[results["variable"].isin(finance_vars)].copy()
print(results_finance)
```

```
Model failures: 0
   regime         country            variable      coef        se         t  \
0      GS       Australia  liquid_liabilities -0.371831  0.342576 -1.085399
1      GS       Australia      bank_loans_gdp -0.303282  0.295894 -1.024971
2      GS       Australia            mcap_gdp  0.212603  0.232890  0.912887
3      GS          Canada  liquid_liabilities -1.514826  0.474964 -3.189353
4      GS          Canada      bank_loans_gdp -0.788429  0.445138 -1.771201
..    ...             ...                 ...       ...       ...       ...
76    FIN  United Kingdom      bank_loans_gdp -0.015603  0.068070 -0.229223
77    FIN  United Kingdom            mcap_gdp  0.039865  0.015359  2.595539
78    FIN   United States  liquid_liabilities -0.148901  0.083706 -1.778848
79    FIN   United States      bank_loans_gdp  0.149359  0.127875  1.168008
80    FIN   United States            mcap_gdp  0.040465  0.017685  2.288171

           p  nobs  maxlags_used
0   0.277745    29             2
1   0.305377    29             2
2   0.361302    29             2
3   0.001426    29             2
4   0.076527    29             2
..       ...   ...           ...
76  0.818696    43             2
77  0.009444    43             2
78  0.075265    43             2
79  0.242803    43             2
80  0.022128    43             2

[81 rows x 9 columns]
```

In [48]:
```python
H1_test = (
    results
    .groupby(["regime", "variable"])["coef"]
    .mean()
    .unstack("regime")
)

print("H1 - Mean coefficients by regime")
print(H1_test)
```

```
H1 - Mean coefficients by regime
regime                    BW       FIN        GS
variable
bank_loans_gdp     -0.302238 -0.056103 -0.535979
liquid_liabilities -0.007194 -0.033775  0.061137
mcap_gdp            0.132139  0.047373  0.163516
```

In [49]:
```python
H2_test = (
    results[results["variable"].isin(finance_vars)]
    .groupby(["regime", "variable"])["coef"]
    .mean()
    .reset_index()
)

print("H2 - Average finance coefficients by regime")
print(H2_test)
```

```
H2 - Average finance coefficients by regime
  regime            variable      coef
0     BW      bank_loans_gdp -0.302238
1     BW  liquid_liabilities -0.007194
2     BW            mcap_gdp  0.132139
3    FIN      bank_loans_gdp -0.056103
4    FIN  liquid_liabilities -0.033775
5    FIN            mcap_gdp  0.047373
6     GS      bank_loans_gdp -0.535979
7     GS  liquid_liabilities  0.061137
8     GS            mcap_gdp  0.163516
```

In [54]:
```python
H3_test = (
    results[(results["regime"] == "BW") &
            (results["variable"].isin(["bank_loans_gdp", "mcap_gdp"]))]
    .groupby("variable")["coef"]
    .agg(["mean", "std", "count"])
)

print("H3 - Bretton Woods: credit vs markets")
print(H3_test)
```

```
H3 - Bretton Woods: credit vs markets
                    mean       std  count
variable
bank_loans_gdp -0.302238  0.343999      9
mcap_gdp        0.132139  1.019018      9
```

In [51]:
```python
H4_test = (
    results[(results["regime"] == "FIN") &
            (results["variable"] == "mcap_gdp")]
)

print("H4 - Financialization: market finance")
print(H4_test[["country", "coef", "p"]])

print("Share negative (FIN, mcap):",
      (H4_test["coef"] < 0).mean())
```

```
H4 - Financialization: market finance
           country      coef         p
56       Australia  0.052057  0.033426
59          Canada  0.012853  0.446123
62         Denmark  0.018977  0.398099
65         Germany  0.086768  0.030018
68          Norway  0.096951  0.038000
71          Sweden  0.051456  0.000557
74     Switzerland  0.026962  0.001230
77  United Kingdom  0.039865  0.009444
80   United States  0.040465  0.022128
Share negative (FIN, mcap): 0.0
```

In [ ]:
```python
def engle_granger_by_country(df, y_level, x_level, entity="country", trend="c", min_obs=25):
    rows = []
    for c in sorted(df[entity].unique()):
        d = df[df[entity] == c].dropna(subset=[y_level, x_level]).sort_values("year")
        if len(d) < min_obs:
            continue

        y = d[y_level].astype(float).to_numpy()
        x = d[x_level].astype(float).to_numpy()

        # coint returns: t_stat, p_value, critical_values
        t_stat, pval, crit = coint(y, x, trend=trend)
        rows.append({
            "country": c,
            "y": y_level,
            "x": x_level,
            "nobs": len(d),
            "EG_tstat": t_stat,
            "p_value": pval,
            "coint_5pct": pval < 0.05
        })

    return pd.DataFrame(rows)


pairs = ["mcap_gdp", "bank_loans_gdp", "liquid_liabilities"]

eg_results = []
for xvar in pairs:
    out = engle_granger_by_country(df_Final, y_level="ln_gdppc", x_level=xvar, trend="c", min_obs=25)
    eg_results.append(out)

eg_results = pd.concat(eg_results, ignore_index=True)

# summary
summary_eg = eg_results.groupby("x")["coint_5pct"].mean().reset_index(name="share_cointegrated_5pct")
print(eg_results)


print(eg_results.sort_values(["x","p_value"]).head(20))
```

```
           country         y                   x  nobs  EG_tstat   p_value  \
0        Australia  ln_gdppc            mcap_gdp   118 -2.949309  0.122752
1           Canada  ln_gdppc            mcap_gdp   118 -1.594901  0.723037
2          Denmark  ln_gdppc            mcap_gdp   127 -1.607282  0.717806
3           France  ln_gdppc            mcap_gdp   127 -0.400644  0.970699
4          Germany  ln_gdppc            mcap_gdp   127 -1.450367  0.779728
5           Norway  ln_gdppc            mcap_gdp   127 -0.947918  0.912288
6         Portugal  ln_gdppc            mcap_gdp   127 -1.913392  0.573200
7            Spain  ln_gdppc            mcap_gdp   118 -2.086631  0.483357
8           Sweden  ln_gdppc            mcap_gdp   127 -1.200631  0.857736
9      Switzerland  ln_gdppc            mcap_gdp   127 -1.669266  0.690790
10  United Kingdom  ln_gdppc            mcap_gdp   127 -1.531910  0.748753
11   United States  ln_gdppc            mcap_gdp   118 -2.005303  0.525820
12       Australia  ln_gdppc      bank_loans_gdp   139 -1.239708  0.847194
13          Canada  ln_gdppc      bank_loans_gdp   139 -1.722689  0.666459
14         Denmark  ln_gdppc      bank_loans_gdp   139 -1.044797  0.894016
15         Finland  ln_gdppc      bank_loans_gdp   139 -1.649428  0.699584
16         Germany  ln_gdppc      bank_loans_gdp   139 -0.902102  0.919898
17           Italy  ln_gdppc      bank_loans_gdp   139 -2.701507  0.199037
18           Japan  ln_gdppc      bank_loans_gdp   139 -2.022342  0.516947
19          Norway  ln_gdppc      bank_loans_gdp   139 -1.259289  0.841684
20          Sweden  ln_gdppc      bank_loans_gdp   139 -1.418188  0.791211
21     Switzerland  ln_gdppc      bank_loans_gdp   139 -0.992608  0.904240
22  United Kingdom  ln_gdppc      bank_loans_gdp   139 -1.759187  0.649326
23   United States  ln_gdppc      bank_loans_gdp   139 -2.128399  0.461534
24       Australia  ln_gdppc  liquid_liabilities   139 -1.455467  0.777870
25          Canada  ln_gdppc  liquid_liabilities   139 -1.418396  0.791138
26         Denmark  ln_gdppc  liquid_liabilities   139  0.582803  0.993130
27         Finland  ln_gdppc  liquid_liabilities   139 -0.444751  0.968040
28          France  ln_gdppc  liquid_liabilities   139 -0.126827  0.982478
29         Germany  ln_gdppc  liquid_liabilities   139 -1.109153  0.880114
30           Italy  ln_gdppc  liquid_liabilities   139 -1.965723  0.546343
31           Japan  ln_gdppc  liquid_liabilities   139 -1.597104  0.722111
32     Netherlands  ln_gdppc  liquid_liabilities   139 -1.910146  0.574852
33          Norway  ln_gdppc  liquid_liabilities   139  0.209773  0.989747
34        Portugal  ln_gdppc  liquid_liabilities   139 -2.878593  0.141954
35           Spain  ln_gdppc  liquid_liabilities   139 -2.270839  0.388190
36          Sweden  ln_gdppc  liquid_liabilities   139  0.157483  0.988960
37     Switzerland  ln_gdppc  liquid_liabilities   139 -1.506525  0.758682
38  United Kingdom  ln_gdppc  liquid_liabilities   139 -1.463221  0.775023
39   United States  ln_gdppc  liquid_liabilities   139 -1.650292  0.699203

    coint_5pct
0        False
1        False
2        False
3        False
4        False
5        False
6        False
7        False
8        False
9        False
10       False
11       False
12       False
13       False
14       False
15       False
16       False
17       False
18       False
19       False
20       False
21       False
22       False
23       False
24       False
25       False
26       False
27       False
28       False
29       False
30       False
31       False
32       False
33       False
34       False
35       False
36       False
37       False
38       False
39       False
           country         y                   x  nobs  EG_tstat   p_value  \
17           Italy  ln_gdppc      bank_loans_gdp   139 -2.701507  0.199037
23   United States  ln_gdppc      bank_loans_gdp   139 -2.128399  0.461534
18           Japan  ln_gdppc      bank_loans_gdp   139 -2.022342  0.516947
22  United Kingdom  ln_gdppc      bank_loans_gdp   139 -1.759187  0.649326
13          Canada  ln_gdppc      bank_loans_gdp   139 -1.722689  0.666459
15         Finland  ln_gdppc      bank_loans_gdp   139 -1.649428  0.699584
20          Sweden  ln_gdppc      bank_loans_gdp   139 -1.418188  0.791211
19          Norway  ln_gdppc      bank_loans_gdp   139 -1.259289  0.841684
12       Australia  ln_gdppc      bank_loans_gdp   139 -1.239708  0.847194
14         Denmark  ln_gdppc      bank_loans_gdp   139 -1.044797  0.894016
21     Switzerland  ln_gdppc      bank_loans_gdp   139 -0.992608  0.904240
16         Germany  ln_gdppc      bank_loans_gdp   139 -0.902102  0.919898
34        Portugal  ln_gdppc  liquid_liabilities   139 -2.878593  0.141954
35           Spain  ln_gdppc  liquid_liabilities   139 -2.270839  0.388190
30           Italy  ln_gdppc  liquid_liabilities   139 -1.965723  0.546343
32     Netherlands  ln_gdppc  liquid_liabilities   139 -1.910146  0.574852
39   United States  ln_gdppc  liquid_liabilities   139 -1.650292  0.699203
31           Japan  ln_gdppc  liquid_liabilities   139 -1.597104  0.722111
37     Switzerland  ln_gdppc  liquid_liabilities   139 -1.506525  0.758682
38  United Kingdom  ln_gdppc  liquid_liabilities   139 -1.463221  0.775023

    coint_5pct
17       False
23       False
```

```
18        False
22        False
13        False
15        False
20        False
19        False
12        False
14        False
21        False
16        False
34        False
35        False
30        False
32        False
39        False
31        False
37        False
38        False
```